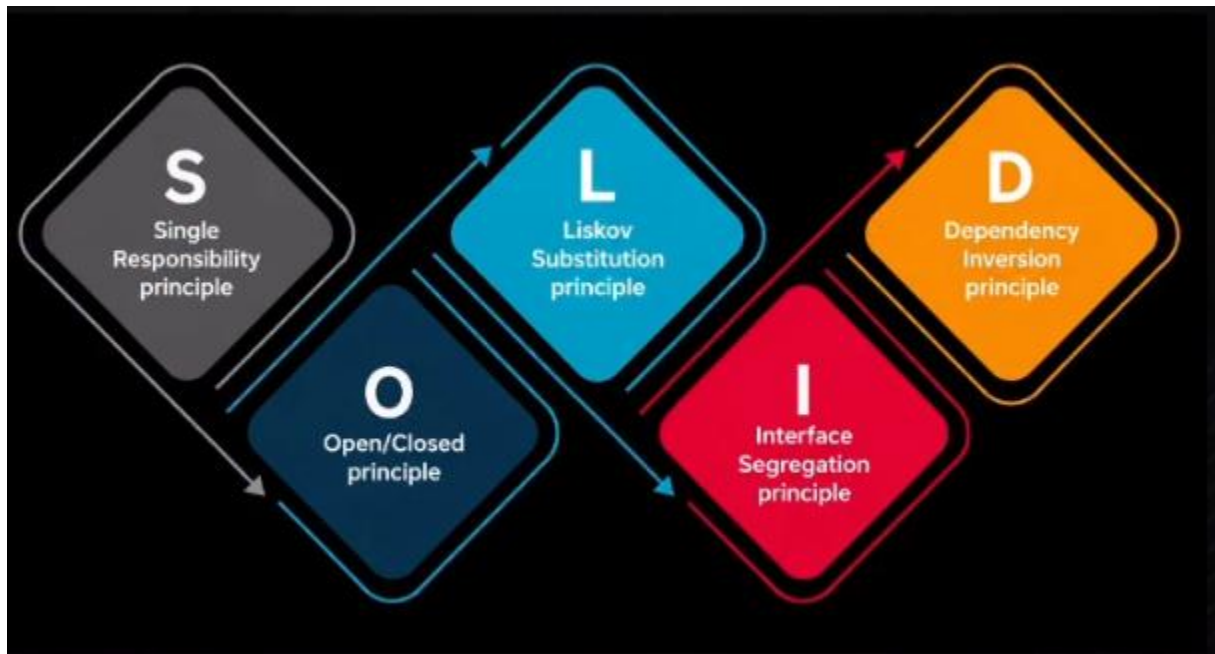


SOLID Principle



Single Responsibility Principle

A class should have one, and only one, reason to change. In other words, each class should focus on a single responsibility or job.

Example: An *Employee* class that handles too many responsibilities can become a “monster class.” For instance, if it calculates salary, manages employee details, and applies tax rules, then a change in the tax regime would require updating the salary-related logic inside the same class.

```
java
// ❌ BAD - "God Class" / "Monster Class"
class Employee {
    String name;
    String id;
    double baseSalary;

    double calculateSalary() {
        // tax + bonus logic here
        return baseSalary * 1.1;
    }

    void generatePayslip() {
        System.out.println("Payslip for " + name);
    }

    void saveToDatabase() {
        System.out.println("Saving " + name + " to DB");
    }
}
```

Solution: Break the large class into smaller, focused classes. This keeps each responsibility separate, makes the code easier to maintain, and reduces the risk that one change will affect unrelated functionality.

Result: The employee-related code becomes cleaner, safer to modify, and easier to understand.

```

java
//  GOOD - Single Responsibility
class Employee {
    String name;
    String id;
    double baseSalary;
}

class SalaryCalculator {
    double calculate(Employee e) {
        return e.baseSalary * 1.1;
    }
}

class PayslipGenerator {
    void generate(Employee e) {
        System.out.println("Payslip for " + e.name);
    }
}

class EmployeeRepository {
    void save(Employee e) {
        System.out.println("Saving " + e.name + " to DB");
    }
}

```

Open-Closed Principle

The Open-Closed Principle states that a class should be **open for extension** but **closed for modification**. This means new functionality should be added by extending existing behavior, not by changing code that is already tested and working.

```

java
// ✗ BAD - violates Open/Closed
class NotificationService {
    void send(String type, String message) {
        if (type.equals("EMAIL")) {
            System.out.println("Sending Email: " + message);
        } else if (type.equals("SMS")) {
            System.out.println("Sending SMS: " + message);
        }
        // new type? edit this same class again
    }
}

```

Add WhatsApp push Notifications

```

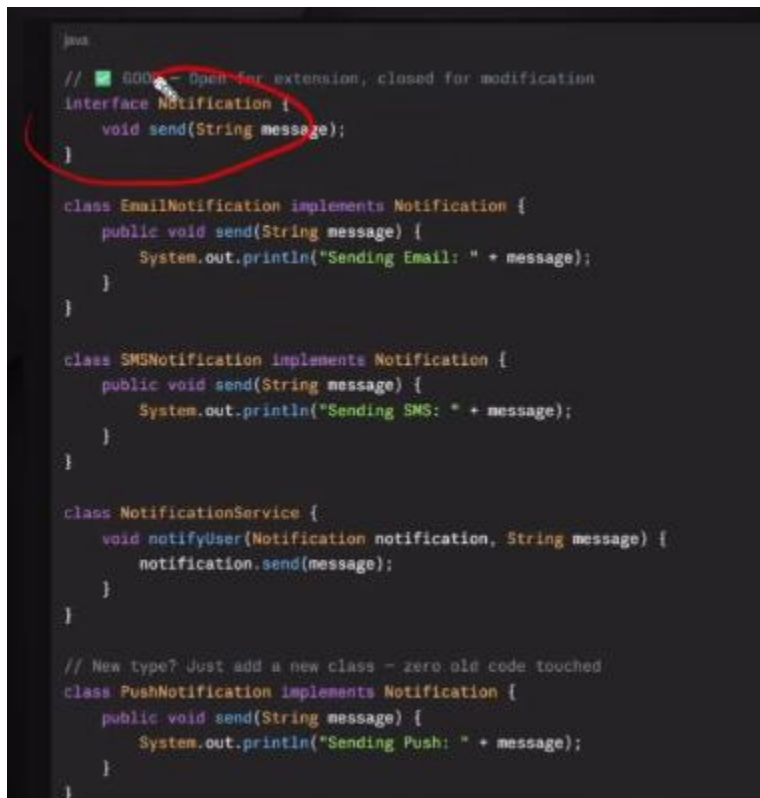
java
// ✗ BAD - violates Open/Closed
class NotificationService {
    void send(String type, String message) {
        if (type.equals("EMAIL")) {
            System.out.println("Sending Email: " + message);
        } else if (type.equals("SMS")) {
            System.out.println("Sending SMS: " + message);
        } else if (type.equals("WHATSAPP")) {
            System.out.println("Sending WhatsApp Push Notification: " + message);
        }
        // new type? edit this same class again
    }
}

```

Example: Suppose an application already supports email and SMS notifications. If we need to add WhatsApp or push notifications, we should avoid adding more *if-else*

conditions to the existing, tested code. Changing that code can introduce bugs and make the system harder to maintain.

Solution: Use polymorphism and define a common notification interface. Each notification type, such as email, SMS, WhatsApp, or push notification, can have its own separate class that implements the interface. To add a new notification type, create a new class instead of modifying the existing code.

A screenshot of a code editor showing Java code. At the top, a comment reads: `// GDD - Open for extension, closed for modification`. Below this is an `interface Notification {` with a method `void send(String message);`. This interface definition is circled in red. Following the interface are three class definitions: `class EmailNotification implements Notification {`, `class SMSNotification implements Notification {`, and `class PushNotification implements Notification {`. Each class has a `public void send(String message)` method that prints a message to the console. Finally, there is a `class NotificationService {` with a `void notifyUser(Notification notification, String message)` method that calls `notification.send(message)`.

```
java
// GDD - Open for extension, closed for modification
interface Notification {
    void send(String message);
}

class EmailNotification implements Notification {
    public void send(String message) {
        System.out.println("Sending Email: " + message);
    }
}

class SMSNotification implements Notification {
    public void send(String message) {
        System.out.println("Sending SMS: " + message);
    }
}

class NotificationService {
    void notifyUser(Notification notification, String message) {
        notification.send(message);
    }
}

// New type? Just add a new class - zero old code touched
class PushNotification implements Notification {
    public void send(String message) {
        System.out.println("Sending Push: " + message);
    }
}
```

Dependency Injection

Dependency Injection (DI) is a design pattern in which an object receives the dependencies it needs from an external source instead of creating them directly. This makes the code easier to test, maintain, and change.

Scenario: Suppose an *EmployeeController* needs an *EmployeeService* to perform employee-related operations.

Without Dependency Injection

```
public class EmployeeController
{
```

```
        EmployeeService service = new EmployeeService();  
    }
```

In this version, the controller creates the *EmployeeService* itself using *new*. This tightly couples the controller to the service implementation.

With Dependency Injection

```
public class EmployeeController  
{  
    private readonly EmployeeService _service;  
    public EmployeeController(EmployeeService service)  
    {  
        _service = service;  
    }  
}
```

In this version, the controller does not create the service object. Instead, ASP.NET Core provides, or *injects*, the *EmployeeService* automatically through the constructor.

UI Building

Use the following Angular CLI commands to create the student management UI project and generate the required components and services.

Create the Angular Project

```
ng new student-management-ui  
cd student-management-ui
```

Generate Components

```
ng generate component components/classes
```

```
ng generate component components/students
```

Generate Services

```
ng generate service services/class  
ng generate service services/student
```

Define the Class Model

The *Class* model defines the structure of a class record in the Angular application. It helps ensure that every class object follows the same format when data is displayed, created, or passed between components and services.

- **classId:** Stores the unique identifier for the class.
- **className:** Stores the name of the class, such as Grade 10 or Computer Science.

```
export interface Class {  
  classId: string;  
  className: string;  
}
```

Define the Student Model

The *Student* model defines the structure of a student record. It makes the student data consistent across the application and helps the components and services work with strongly typed data.

- **studentId:** Stores the unique identifier for the student.
- **firstName:** Stores the student's first name.
- **lastName:** Stores the student's last name.
- **email:** Stores the student's email address.
- **classId:** Links the student to a specific class.

```
export interface Student {  
  studentId: string;  
  firstName: string;  
  lastName: string;  
  email: string;  
  classId: string;  
}
```

Define the Class Service

The *ClassService* is responsible for communicating with the backend API for class-related operations. It uses Angular's *HttpClient* to retrieve, create, update, and delete class records.

Imports and Service Setup

```
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Class } from '../../models/class.model';

@Injectable({
  providedIn: 'root'
})
```

Service Class and API URL

```
export class ClassService {
  private apiUrl = 'https://localhost:7094/api/classes';
  constructor(private http: HttpClient) { }
```

Read Classes

```
  getClasses() {
    return this.http.get<Class[]>(this.apiUrl);
  }
```

Create a Class

```
  addClass(classData: { className: string }) {
    return this.http.post<Class>(this.apiUrl, classData);
  }
```

Update a Class

```
  updateClass(classId: string, classData: { className: string })
  {
    const url = `${this.apiUrl}/${classId}`;
    return this.http.put<Class>(url, classData);
  }
```


Delete a Class

```
deleteClass(classId: string) {  
    const url = `${this.apiUrl}/${classId}`;  
    return this.http.delete(url);  
}  
}
```

Define the Student Service

The *StudentService* handles communication between the Angular application and the backend student API. It uses Angular's *HttpClient* to retrieve, create, update, and delete student records.

Imports and Service Setup

```
import { HttpClient } from '@angular/common/http';  
import { Injectable } from '@angular/core';  
import { Student } from '../../models/student.model';
```

Service Class and API URL

```
@Injectable({  
    providedIn: 'root'  
})  
  
export class StudentService {  
    private apiUrl = 'https://localhost:7094/api/students';  
    constructor(private http: HttpClient) { }  
}
```

Read Students

```
getStudents() {  
    return this.http.get<Student[]>(this.apiUrl);  
}
```

Create a Student

```
addStudent(studentData: Student) {  
    return this.http.post<Student>(this.apiUrl, studentData);  
}
```

Update a Student

```
updateStudent(studentId: string, studentData: Student) {  
    const url = `${this.apiUrl}/${studentId}`;  
    return this.http.put<Student>(url, studentData);  
}
```

Delete a Student

```
deleteStudent(studentId: string) {  
    const url = `${this.apiUrl}/${studentId}`;  
    return this.http.delete(url);  
}  
}
```

Summary: This service keeps all student-related API calls in one place, making the components cleaner and easier to maintain.

Configure Application Routes

The route configuration defines how users navigate between the main pages of the Angular application. In this setup, the default route opens the classes page, while separate routes are provided for the classes and students components.

Imports

```
import { Routes } from '@angular/router';  
  
import { ClassesComponent } from  
'./components/classes/classes.component';  
  
import { StudentsComponent } from  
'./components/students/students.component';
```

Route Definitions

```
export const routes: Routes = [  
    { path: '', redirectTo: 'classes', pathMatch: 'full' },  
    { path: 'classes', component: ClassesComponent },  
    { path: 'students', component: StudentsComponent }  
];
```

Build the Application Layout

The main layout provides a simple navigation menu for the Student Management System. Users can switch between the *Classes* and *Students* pages, while the routed component is displayed inside the router outlet.

Application Shell

```
<h1>Student Management System</h1>

<nav>

  <ul>

    <li routerLinkActive="active"><a
routerLink="/classes">Classes</a></li>

    <li routerLinkActive="active"><a
routerLink="/students">Students</a></li>

  </ul>

</nav>

<router-outlet></router-outlet>
```